Worksheet: Understanding the YOLOv8 Pose Predictor Head

This worksheet guides students through the internal logic of a modern pose estimation model. Rather than treating the model as a black box, we will:

- Run inference once.
- Inspect th outputs of the predictor head.
- Project numerical predictions back into image space for visualisation.

Step 0 — Install Dependencies and Import Libraries

Set up the environment required to run YOLOv8 pose estimation in Google Colab.

In [1]:
```
!pip install ultralytics opencv-python --quiet
import cv2
import numpy as np
from ultralytics import YOLO
from google.colab.patches import cv2_imshow
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/1.2 MB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╸━ 1.1/1.2 MB 31.0 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 15.0 MB/s eta 0:00:00
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
```

Step 1 — Load a Sample Image

Provide a fixed input so that all students observe the same model behaviour.

In [2]:
```
!wget -q https://ultralytics.com/images/bus.jpg -O test.jpg

image = cv2.imread("test.jpg")
cv2_imshow(image)
```


M1 SOL/SEVILLA
EMT MADRID
Un minibús 100% eléctrico sí es plan
eléctricamEMTe
cero emisiones

Step 2 — Load a Pretrained YOLOv8 Pose Model

Introduce the concept of a pose predictor head, which outputs structured numerical predictions rather than images.

In [3]:
```
model = YOLO("yolov8n-pose.pt")  # lightweight pose model
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo
v8n-pose.pt to 'yolov8n-pose.pt': 100% ━━━━━━━━━━━━ 6.5MB 75.0MB/s 0.1s
```

Step 3 — Run Inference

Perform a forward pass through the network and obtain the raw predictor head outputs.

In [4]:
```
results = model("test.jpg")[0]

print("Bounding boxes:", results.boxes.xyxy.shape)
print("Keypoints:", results.keypoints.data.shape)
```

```
image 1/1 /content/test.jpg: 640x480 4 persons, 558.7ms
Speed: 11.8ms preprocess, 558.7ms inference, 51.7ms postprocess per image at sh
ape (1, 3, 640, 480)
Bounding boxes: torch.Size([4, 4])
Keypoints: torch.Size([4, 17, 3])
```

Step 4 — Visualise Predictor Head Output in Image Space

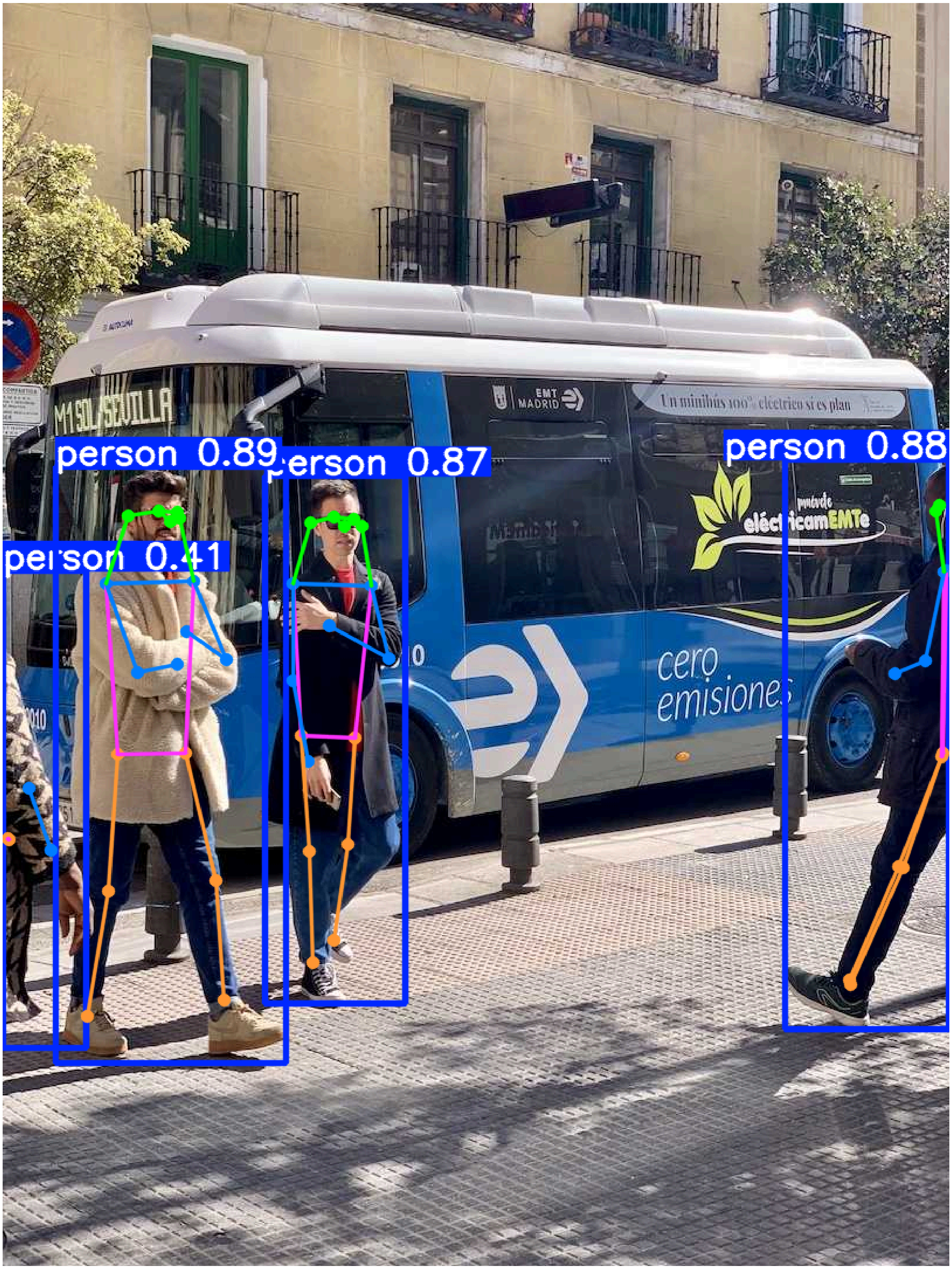
Project numerical predictions back into the image for human interpretation.

In [5]:
```
# Base heatmap
heatmap = np.zeros(image.shape[:2], dtype=np.float32)
h, w = heatmap.shape

for person in results.keypoints.data:
    for k in range(17):
        x, y, conf = person[k]
        x = int(round(float(x)))
        y = int(round(float(y)))
        conf = float(conf)

        if conf < 0.1:
            continue
        if 0 <= x < w and 0 <= y < h:
            heatmap[y, x] += conf

# Spatial smoothing → real heatmap
```

```python
heatmap = cv2.GaussianBlur(heatmap, (0, 0), sigmaX=18, sigmaY=18)

# Robust normalisation (ignore background)
non_zero = heatmap[heatmap > 0]
vmax = np.percentile(non_zero, 99) if non_zero.size else 1.0

heatmap = np.clip(heatmap, 0, vmax)
heatmap = (heatmap / vmax * 255).astype(np.uint8)

# Colour map and overlay
heatmap_color = cv2.applyColorMap(heatmap, cv2.COLORMAP_TURBO)
overlay = cv2.addWeighted(image, 0.4, heatmap_color, 0.6, 0)

cv2_imshow(overlay)
```

In [6]:
```
annotated = results.plot()  # returns an image with drawings

# Display the annotated image in Colab
```

```
cv2_imshow(annotated)
```



Final Step — Run Pose Estimation on Your Own Image

You can upload your own image and run the pose estimator on it.

In [7]:
```
from google.colab import files

uploaded = files.upload()

# Get the uploaded file name
image_path = list(uploaded.keys())[0]
print("Uploaded file:", image_path)

# Load YOLOv8 Pose model
model = YOLO("yolov8n-pose.pt")  # lightweight pose model

# Run inference on the uploaded image
results = model(image_path)[0]

# Read image and draw keypoints/skeleton
image = cv2.imread(image_path)
annotated = results.plot()  # returns an image with pose drawings

cv2_imshow(image)
# Display the annotated image in Colab
cv2_imshow(annotated)
```

Browse... Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving 903b4728gw1esl7n20543j21ao0q9tgr.jpg to 903b4728gw1esl7n20543j21ao0q9tg
r.jpg
Uploaded file: 903b4728gw1esl7n20543j21ao0q9tgr.jpg

image 1/1 /content/903b4728gw1esl7n20543j21ao0q9tgr.jpg: 384x640 16 persons, 24
5.2ms
Speed: 11.4ms preprocess, 245.2ms inference, 1.3ms postprocess per image at sha
pe (1, 3, 384, 640)
```

In [7]: